

Summer Training Program WEEK 1 - DAY 2

Topics discussed:

- IDE
- Dunders/magic variables
- Process
- Threads
- Representation of data in memory
- Types of variables/containers - single valued and multi valued
- Types of multivalued containers - homogenous and heterogenous
- Basic python programs
- Introduction to id()/hashcode
- Representation of list in memory (python data types)

IDE- Integrated Development Environment

Two mainly used IDEs for python coding are:

1. Visual Studio Code:
Highly customizable and also supports other languages
2. PyCharm:
Dedicated python IDE used by many mnc's

Dunders

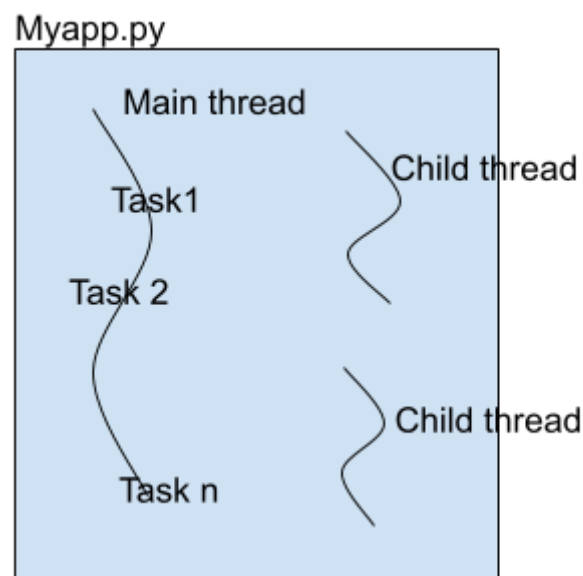
Dunders are special variables in python which begin with ‘__’ and also end with a ‘__’.

Eg: `__name__`, `__main__` are dunder.

By using Dunders we can customize the behaviour of our objects (will be discussed in later classes).

Process and Threads

- A program in execution is called a process.
- A process contains threads which execute the tasks associated with the process step by step.
- A process may contain multiple threads but it always contains one thread which is called “Main Thread”.
- All the other threads contained in a process may be called child threads.



Eg of multiple threads:

In the bookmyshow application when multiple people are booking tickets at the same time on different threads but same process

Container/Object/Model

It is basically what holds the values/data in python.

There may be single valued or multivalued containers in python.

1. Single valued containers:

It holds only one single value

Eg: username, email, age etc

All these variables only have a single value.

2. Multivalued containers:

They can contain many values and they are also of two types:

A. Homogenous : The data stored is all of the same datatype.

Eg: A photodump on instagram, contains only photographs.

B. Heterogeneous: The data stored is of different types.

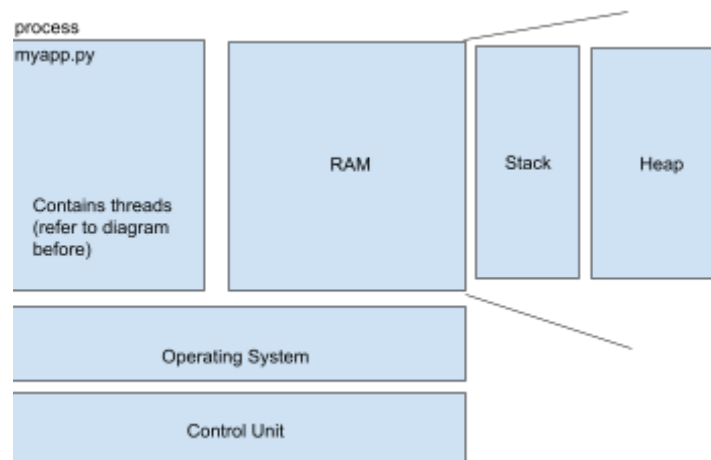
Eg: A post on Instagram with photos, videos and gifs etc.

Representations of containers in memory(RAM):

- All the variables being currently worked on are kept in ram while process execution.
- These are kept in a **Stack** and a **Heap**.
- Stacks contain hashcodes of the containers.
- And the heap stores the actual containers.

NOTE: The address of containers can be accessed by using the `id()` function. It returns the hashcode of the variable.

Simple representation:



SINGLE VALUED CONTAINERS

CODE:

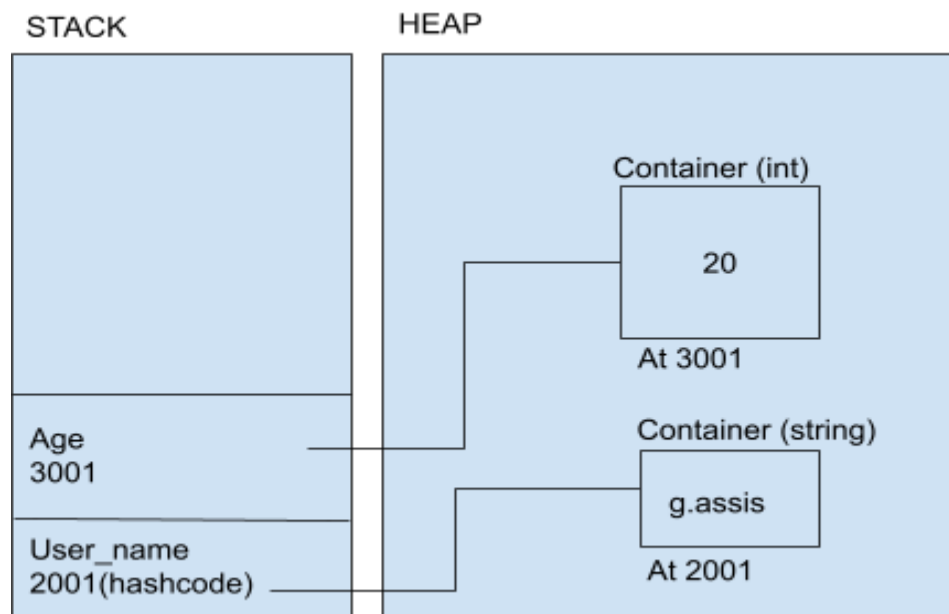
```
1 user_name= 'g.assis'
2 age = 20
3
4
5 print('Hashcode of user_name:', id(user_name))
6 print('Hashcode of age:', id(age))
```

1. The variable user_name is a referential variable and the value 'g.assis' is an actual literal/data, both of the variables use a single valued container.
2. The 1st and 2nd statement is a creational statement (linked to model).
3. The 5th and 6th statements are output statements (linked to view).

OUTPUT:

```
Hashcode of user_name: 3172404407264
Hashcode of age: 140719719187976
```

MEMORY REPRESENTATION:



NOTE: Classes in c++ are nothing but customized/user defined containers.

Important facts:

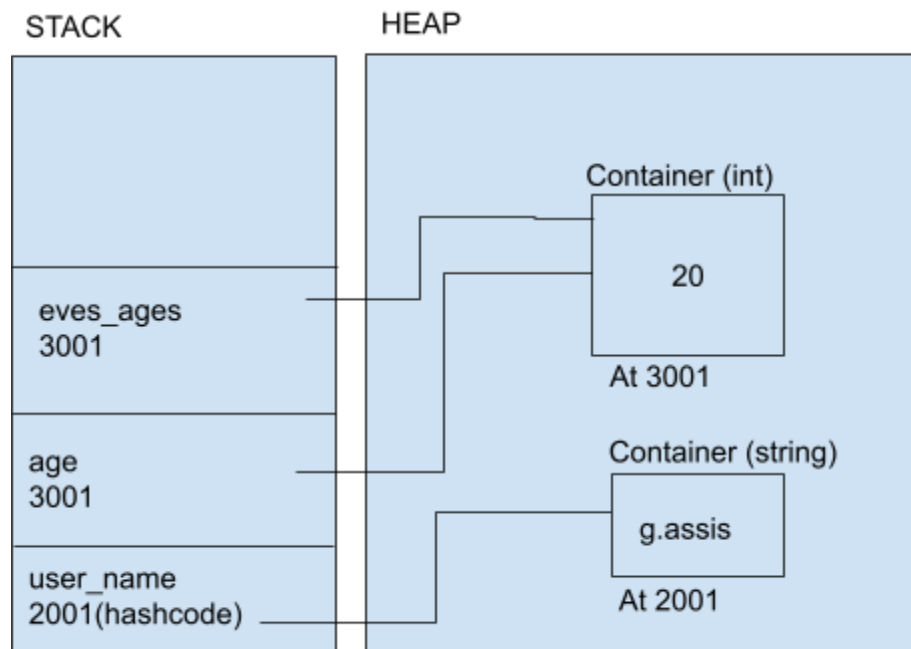
1. If we want to assign two variables to the same value i.e.

Age =20

eves_age=20

They will not have separate containers. Both of the variables in stack will point to the same variable i.e. store the same hascode.

```
Hashcode of age: 140719977203208
Hashcode of eves_age: 140719977203208
```



2. If we want to delete a variable There may be two cases:

- a. If the container is only pointed to by one value.

eg: `user_name` in the above example, no other variable is pointing to the same username.

In this case, the container is deleted from the heap and the variable is also deleted from the stack.

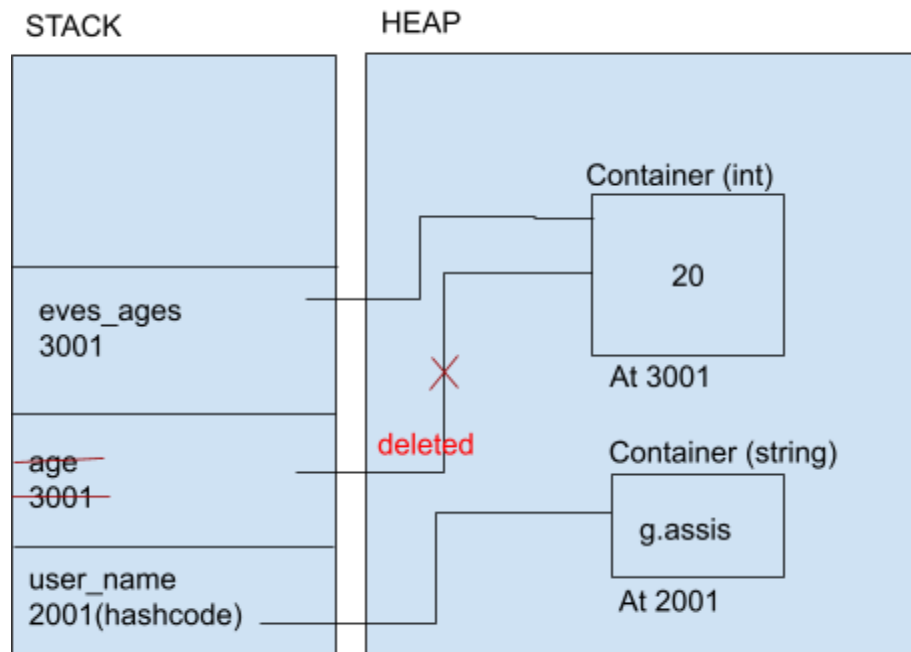
- b. If the container is shared by two or more variable:

eg: `age` and `eves_age` in the above example

Say we want to delete `age`, now only the reference of `age` to the container will be deleted.

The container will not be deleted because it stores the value of `eves_age` in it too.

After deletion :



MULTIVALUED CONTAINERS:

Multivalued containers can store more than one values at once, for this purpose various data structures are used:

1. List
2. Set
3. Tuple
4. Dictionary

Multivalued containers are further classified into two parts:

1. Homogenous:
All the values/data stored is of same data type
Eg : marks={70, 80, 75} #homogenous list
2. Heterogenous:
Data may has more than one data type.
Eg: adams_marks = {70, 'B', 80, 'A'} #heterogenous list

NOTE:

We cannot perform operations such as searching, sorting and other algorithms on heterogeneous containers because conditional constructs can only be applied on variables with same data type.

These can still be used for data dump.

Multivalued containers in memory:

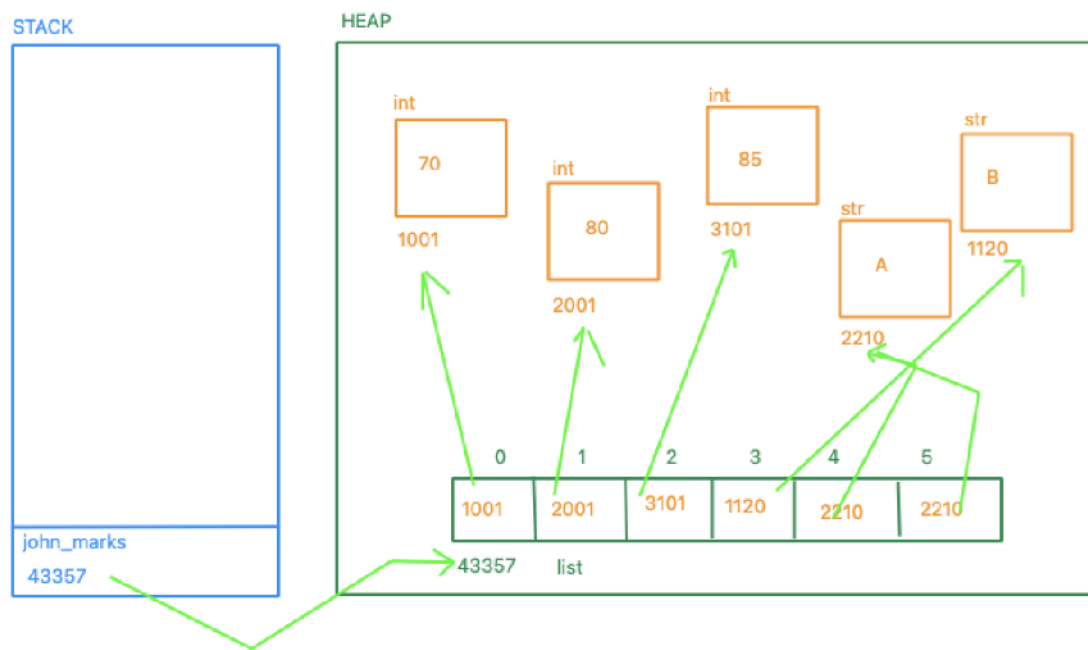
CODE:

```
3  johns_marks = [70,80,85,'A','B']
4
5  print('Hashcode of johns_marks:', id(johns_marks), type(johns_marks))
```

OUTPUT:

```
mer training/week 1/day2/2.py"
Hashcode of johns_marks: 1737077710976 <class 'list'>
```

MEMORY REPRESENTATION:



Important facts:

1. A list in the heap does not store the data values itself. It stores the hashcodes of the single valued containers.
2. From this diagram, we can conclude that Multivalued containers are comprised of many singlevalued containers.
3. In this previous example, we've used a list, Indexing of list is done automatically in python.
4. For multivalued containers, we can use various data types and even nest them.
5. The concept of same data and shared containers still applies.
6. **Multivalued containers are not shared even when the value is the same.**

Introduction to data types:

1. List

It can be created in python using [] - square brackets

Lists are mutable, hence they can be used when both read and write operation are to be performed.

Indexing is done by default in a list and goes from 0 to n-1 where n is the number of elements.

Lists support duplicacy/duplicate data.

2. Tuple

It can be created in python using ()- round brackets

Tuples are immutable, hence they can be used when only read operation is to be performed.

Tuples are also indexed similarly.

Tech Lingo:

The words 'MODEL', 'CONTAINER', 'OBJECT' all refer to the same entity just in different use cases/situations.

MODEL is used during planning.

CONTAINER is used while understanding memory allocation.

OBJECT is used when programming/actual implementation.

INTERVIEW QUESTIONS:

For C++

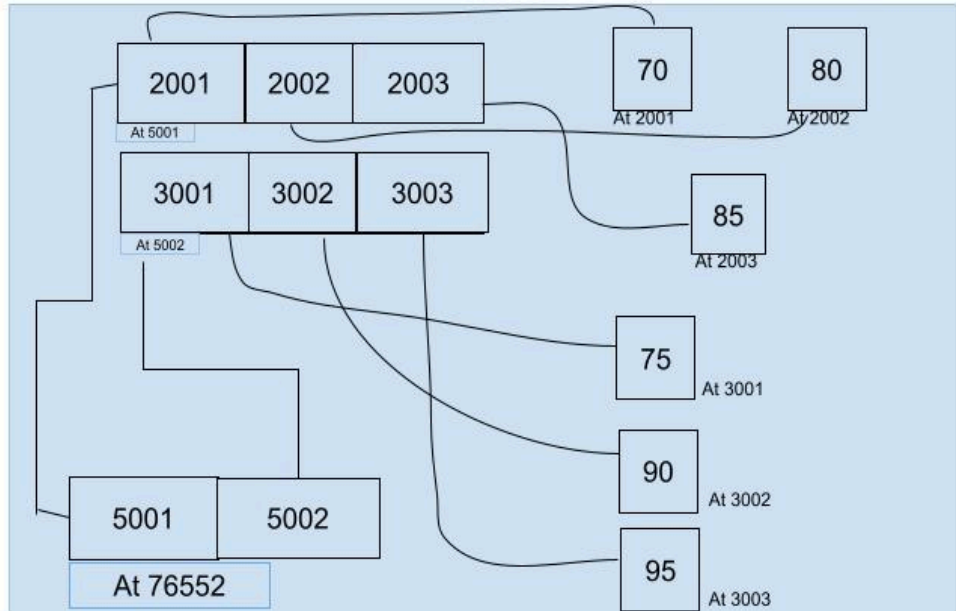
- Dangling pointers
- Memory leaks
- Deep copy and shallow copy (very imp.)

NOTE:

The hash code we studied and the actual address on the RAM are different because python does not give direct memory access unlike C++, this is encryption.

ASSIGNMENT: Draw the memory representation/utilization of a list in a list.

```
marks = {  
    {70,80,85}  
    {75,90,95}  
}
```



GURASSIS KAUR
g.assiskaur@gmail.com